Parallel Processing Lab

MPI Assignment: Selected Problems with Lab-Exam and Viva Notes

Section A and B - Problems 2, 3, 4, 5, 7, 8, 10, 11, 15, 16, 17, 18

Problem	Topic	Run with
2	Matrix row distribution and row sums (Send/Recv)	-np 4
3	Ring topology and timing	-np 4 (any P >= 2)
4	Ping-pong: latency and bandwidth	-np 2
5	Distributed matrix multiplication	-np 2
7	Nonblocking Isend/Irecv vs blocking	-np 2
8	Deadlock and how to avoid it	-np 2
10	Producer-consumer	-np 3 (any P >= 2)
11	Broadcast of configuration parameters	-np 4 (any P)
15	Trapezoidal rule integration	-np 4 (any P)
16	Prefix sum with MPI_Scan	-np 4 (any P)
17	All-to-all communication	-np 4 (any P)
18	Global histogram with MPI_Allreduce	-np 4

Each problem contains: the full question with sample input/output, complete C/MPI code, a step-by-step explanation, the MPI calls used, viva questions with model answers, and common mistakes.

Compile any program: `mpicc file.c -o file` (add `-lm` for Problem 15). **Run:** `mpirun -np N ./file` (or `mpiexec`).

Output order: when different processes print, the order of lines is not guaranteed, so your screen may differ from the sample while still being correct.

Core MPI Concepts You Must Know

Read this page before the viva. Every problem builds on these ideas.

Term	Meaning
SPMD model	All processes run the same program. The rank (MPI_Comm_rank) decides what each process does.
Communicator	A group of processes that can communicate. MPI_COMM_WORLD contains all of them. Size = MPI_Comm_size, id = rank (0 to size-1).
Point-to-point	One sender, one receiver: MPI_Send / MPI_Recv. A receive matches a send by source, tag and communicator.
Collective	All processes in the communicator call it: Bcast, Scatter, Gather, Reduce, Allreduce, Scan, Alltoall, Barrier. No tags.
Blocking vs nonblocking	Blocking calls return when the buffer is safe to reuse. Nonblocking (Isend/Irecv) return immediately; you must MPI_Wait before using the buffer.
Deadlock	Processes wait for each other forever, typically both in MPI_Send. Avoid with Sendrecv, ordering, or nonblocking calls.
Eager vs rendezvous	Small messages are copied to a buffer and sent at once (eager); large ones need the receiver to be ready (rendezvous). Never rely on eager buffering for correctness.
Latency and bandwidth	Time = latency + size / bandwidth. Small messages are latency-bound; large messages are bandwidth-bound.
MPI_Wtime	Wall-clock time in seconds. Repeat the measurement and average; use a barrier to start all ranks together.
Speed-up and efficiency	Speed-up = T(serial) / T(parallel). Efficiency = speed-up / P. Communication overhead limits both.

Collective operations at a glance

Call	Data movement	Used in
MPI_Bcast	root -> everyone (same data)	Problem 11 (and B in Problem 5)
MPI_Scatter / Scatterv	root -> everyone (different pieces)	-
MPI_Gather / Gatherv	everyone -> root	Problems 15, 16 (reporting)
MPI_Reduce	everyone -> root, combined with an operation	Problems 15, 17
MPI_Allreduce	everyone -> everyone, combined	Problem 18
MPI_Scan	prefix reduction, each rank gets partial result	Problem 16
MPI_Alltoall	everyone -> everyone, distinct block per pair	Problem 17

Corrections made to the supplied solution file

- **Problem 3:** now uses prev/next for printing (no hard-coded 'P3') and adds the timing part for different message sizes that the question asks for.
- **Problem 4:** now loops over all required message sizes (64 B to 1 MB) and reports bandwidth.
- **Problem 5:** the supplied loop `local_C[i] += local_A[j]*B[j][i]` is actually correct (the guide's warning about it is mistaken); only the variable names were changed for clarity, B is broadcast, and the exact sample output is printed.
- **Problem 7:** adds the blocking baseline so the two methods can really be compared; both ranks print their received vectors.
- **Problem 8:** adds the unsafe version (run with the argument 'unsafe') besides the Sendrecv fix.
- **Problem 10:** works for any number of consumers and prints the final completion message.

- **Problem 15:** handles n not divisible by P, prints partial integrals, exact value and error, and repeats for several n.
- **Problem 16:** adds the manual point-to-point version and compares it with MPI_Scan.
- **Problem 17:** works for any P and adds the timing experiment.
- **Problem 18:** the sample sheet says 20 elements, but the data has 16; the program prints the real total.

Problem 2: Matrix Row Distribution and Row Sums

1. Question (from the assignment)

The root process distributes different rows of a matrix to worker processes using MPI_Send. Workers calculate the sum of their assigned rows and return the results using MPI_Recv (the root receives them).

Sample Input

```
Number of processes: 4
Matrix:
 1  2  3  4
 5  6  7  8
 9 10 11 12
13 14 15 16
Each process receives one row.
```

Sample Output

```
P0: Row = 1 2 3 4 Sum = 10
P1: Row = 5 6 7 8 Sum = 26
P2: Row = 9 10 11 12 Sum = 42
P3: Row = 13 14 15 16 Sum = 58
Total matrix sum = 136
```

2. Solution code

Compile / run: `mpicc p2.c -o p2 && mpirun -np 4 ./p2`

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 4) {                        /* one row per process, 4x4 matrix */
        if (rank == 0) printf("Run with exactly 4 processes.\n");
        MPI_Finalize();
        return 1;
    }

    int matrix[4][4] = {{1,2,3,4}, {5,6,7,8}, {9,10,11,12}, {13,14,15,16}};
    int local_row[4];

    if (rank == 0) {
        for (int i = 0; i < 4; i++) local_row[i] = matrix[0][i];   /* root keeps row 0 */
        for (int p = 1; p < size; p++)                             /* sends row p to P p */
            MPI_Send(matrix[p], 4, MPI_INT, p, 0, MPI_COMM_WORLD);
    } else {
        MPI_Recv(local_row, 4, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }

    int sum = 0;
    for (int i = 0; i < 4; i++) sum += local_row[i];
    printf("P%d: Row = %d %d %d %d Sum = %d\n", rank,
           local_row[0], local_row[1], local_row[2], local_row[3], sum);

    if (rank == 0) {
        int total = sum, recv_sum;
        for (int p = 1; p < size; p++) {
            MPI_Recv(&recv_sum, 1, MPI_INT, p, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            total += recv_sum;
        }
        printf("Total matrix sum = %d\n", total);
    } else {
        MPI_Send(&sum, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}
```

3. Step-by-step explanation (for the lab exam)

- **Idea:** classic master-worker pattern. Root owns the whole matrix; every process works on exactly one row (data decomposition by rows).
- **Distribute:** root keeps row 0 and sends row p to process p with `MPI_Send(matrix[p], 4, MPI_INT, p, tag 0, ...)`. In C, `matrix[p]` is the address of row p, and rows are contiguous in memory, so sending 4 ints is correct.
- **Receive:** each worker posts a matching `MPI_Recv` from source 0 with the same tag 0 into `local_row`.
- **Compute:** every process (including root) sums its own 4 numbers - this part runs in parallel.
- **Return results:** workers send their partial sum with tag 1. Root receives from each worker in rank order and adds to its own sum to get 136. Different tags (0 for data, 1 for results) keep the two kinds of messages from being confused.
- **Check the numbers:** 10 + 26 + 42 + 58 = 136.
- **Output order:** the four 'P? : Row ...' lines are printed by different processes, so their order on screen is not guaranteed. The total line is always last because root prints it after all receives.

4. MPI calls used

- `MPI_Init / MPI_Finalize` - start and end the MPI environment.
- `MPI_Comm_rank / MPI_Comm_size` - my id and number of processes in MPI_COMM_WORLD.
- `MPI_Send(buf, count, type, dest, tag, comm)` and `MPI_Recv(buf, count, type, source, tag, comm, status)` - blocking point-to-point.

5. Viva questions and model answers

Q. Why does every process call MPI_Comm_rank?

Every process runs the same program (SPMD). The rank is the only thing that lets a process know whether it is root or a worker, so the code branches on it.

Q. What do the arguments of MPI_Send mean?

Address of data, number of elements, MPI datatype, destination rank, message tag, communicator.

Q. What is a tag used for?

To label messages so a receive can match the intended one. Here tag 0 = row data, tag 1 = partial sum.

Q. What happens if you run with 3 or 6 processes?

With 3, row 3 is never processed so the total is wrong. With 6, root would send matrix[4] and matrix[5], which are out of bounds (undefined behaviour). That is why the program checks size == 4.

Q. Can you do this with a collective?

Yes: MPI_Scatter to distribute rows, then MPI_Reduce with MPI_SUM (or MPI_Gather). The lab asks for point-to-point to show how collectives are built from sends and receives.

Q. Is MPI_Send guaranteed to be non-blocking for 4 integers?

No. A standard send may buffer small messages, but the standard does not guarantee it. This program is safe anyway because root sends first and every worker is already waiting in a receive.

6. Common mistakes (examiners look for these)

- Running with a different number of processes than the matrix needs.
- Sending `matrix[i][0]` (a value) instead of the address `matrix[i]` or `&matrix[i][0]`.
- Mismatched tag, count or datatype between Send and Recv - the program hangs or gives garbage.
- Forgetting that root also has a row to process.

Problem 3: Ring Topology Communication

1. Question (from the assignment)

Implement a ring topology in which every MPI process sends a message to its next process and receives a message from its previous process. Measure the communication time for different message sizes.

Sample Input

```
Number of processes = 4
Initial value at P0 = 100
Communication: P0 -> P1 -> P2 -> P3 -> P0
```

Sample Output

```
P0 sends 100 to P1
P1 receives 100 from P0
P1 sends 100 to P2
P2 receives 100 from P1
P2 sends 100 to P3
P3 receives 100 from P2
P3 sends 100 to P0
P0 receives 100 from P3
Ring communication completed.
```

2. Solution code

Compile / run: `mpicc p3.c -o p3 && mpirun -np 4 ./p3` (works for any process count of 2 or more)

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        if (rank == 0) printf("Need at least 2 processes.\n");
        MPI_Finalize();
        return 1;
    }

    int next = (rank + 1) % size;             /* wraps last -> 0   */
    int prev = (rank - 1 + size) % size;      /* wraps 0 -> last   */

    /* ---------- Part A: pass the value 100 once around the ring ---------- */
    int msg = 100;
    if (rank == 0) {
        printf("P0 sends %d to P%d\n", msg, next);
        MPI_Send(&msg, 1, MPI_INT, next, 0, MPI_COMM_WORLD);
        MPI_Recv(&msg, 1, MPI_INT, prev, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("P0 receives %d from P%d\n", msg, prev);
        printf("Ring communication completed.\n");
    } else {
        MPI_Recv(&msg, 1, MPI_INT, prev, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("P%d receives %d from P%d\n", rank, msg, prev);
        printf("P%d sends %d to P%d\n", rank, msg, next);
        MPI_Send(&msg, 1, MPI_INT, next, 0, MPI_COMM_WORLD);
    }
    fflush(stdout);
    MPI_Barrier(MPI_COMM_WORLD);

    /* ---------- Part B: time one full trip for different message sizes ---------- */
    int sizes[] = {4, 64, 1024, 16384, 262144, 1048576};      /* bytes */
    int nsizes = sizeof(sizes) / sizeof(sizes[0]);
    int reps = 20;
    char *buf = malloc(1048576);
    memset(buf, 1, 1048576);

    for (int k = 0; k < nsizes; k++) {
        int n = sizes[k];
        MPI_Barrier(MPI_COMM_WORLD);
        double t0 = MPI_Wtime();
```

```
        for (int r = 0; r < reps; r++) {
            if (rank == 0) {
                MPI_Send(buf, n, MPI_CHAR, next, 1, MPI_COMM_WORLD);
                MPI_Recv(buf, n, MPI_CHAR, prev, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            } else {
                MPI_Recv(buf, n, MPI_CHAR, prev, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                MPI_Send(buf, n, MPI_CHAR, next, 1, MPI_COMM_WORLD);
            }
        }
        double t1 = MPI_Wtime();
        if (rank == 0)
            printf("Size %8d bytes : %.9f s per full ring trip\n", n, (t1 - t0) / reps);
    }

    free(buf);
    MPI_Finalize();
    return 0;
}
```

3. Step-by-step explanation (for the lab exam)

- **Neighbours:** `next = (rank+1) % size` makes the last process point back to rank 0; `prev = (rank-1+size) % size` makes rank 0's previous process the last rank. The `+size` avoids a negative number when rank = 0.
- **Why rank 0 sends first:** every other process starts with a blocking receive, so nobody can send until the token reaches them. Rank 0 breaks the circular wait by sending first and receiving last. If every process did Send-then-Recv, the ring could deadlock.
- **Token flow:** P0 -> P1 -> ... -> P(size-1) -> P0. The value 100 is never modified; each process just forwards it. The prev/next variables mean no process number is hard-coded (the original solution printed 'P3' literally, which is only right for 4 processes).
- **Timing:** `MPI_Wtime()` returns wall-clock seconds. A `MPI_Barrier` before timing makes all processes start together. We repeat the trip 20 times and divide to reduce noise, then repeat for message sizes from 4 bytes to 1 MB.
- **What to say about results:** for small messages the time is dominated by latency (fixed start-up cost per hop), so time per trip is almost flat; for large messages it grows roughly linearly with size because bandwidth is the limit. Total ring time is about (number of hops) x (time per hop) = P x (latency + size/bandwidth).

4. MPI calls used

- `MPI_Send / MPI_Recv` - blocking point-to-point.
- `MPI_Barrier` - synchronise all processes before timing.
- `MPI_Wtime` - high-resolution wall-clock timer (seconds, double).

5. Viva questions and model answers

Q. Why is the ring deadlock-free in your program?

Rank 0 starts with a send while all others start with a receive, so there is always one process able to make progress. The chain of dependencies ends at rank 0's final receive, so there is no cycle of processes all waiting.

Q. What if all processes did MPI_Send first and then MPI_Recv?

It might work for tiny messages because of internal buffering, but for large messages every process blocks in MPI_Send waiting for a receive that nobody has posted: deadlock. A safe alternative is MPI_Sendrecv.

Q. How do you compute the previous rank for process 0?

(rank - 1 + size) % size gives size-1. Adding size first prevents a negative modulo in C.

Q. Why do you run many repetitions?

A single measurement of a microsecond-scale operation is noisy (scheduling, cache warm-up, first-call costs). Averaging many repetitions gives a stable value.

Q. How does time change with message size and why?

Small messages: latency-bound, nearly constant. Large messages: bandwidth-bound, time ~ size / bandwidth. Time = latency + size / bandwidth is the simple model.

Q. What is the difference between wall-clock time and CPU time?

MPI_Wtime is elapsed real time, including waiting for messages. CPU time counts only time the processor executes instructions, so it would hide communication waiting.

6. Common mistakes (examiners look for these)

- Hard-coding the process count (e.g. printing 'P3') instead of using prev/next/size.
- Every process sending first - deadlock risk for large messages.
- Timing a single run without a barrier or repetitions.
- Using MPI_INT with a byte count (count is in elements, not bytes).

Problem 4: Ping-Pong: Latency and Bandwidth

1. Question (from the assignment)

Implement a ping-pong program between two MPI processes using MPI_Send and MPI_Recv. Test message sizes from a few bytes to several megabytes and analyze communication latency and bandwidth.

Sample Input

```
Number of processes: 2
Number of iterations = 5
Message size = 1024 bytes
```

Sample Output

```
Message size : 1024 bytes
Iterations : 5
Total round trips : 5
Average round-trip : 0.000018 seconds
Average one-way : 0.000009 seconds

Repeat the experiment for 64 bytes, 256 bytes, 1 KB, 4 KB, 16 KB, 64 KB, and 1 MB.
```

2. Solution code

Compile / run: `mpicc p4.c -o p4 && mpirun -np 2 ./p4`

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define ITER 5          /* use 5 to match the sample; use 1000 for stable measurements */

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2) {
        if (rank == 0) printf("Run with exactly 2 processes.\n");
        MPI_Finalize();
        return 1;
    }

    int sizes[] = {64, 256, 1024, 4096, 16384, 65536, 1048576};   /* bytes */
    int nsizes = sizeof(sizes) / sizeof(sizes[0]);
    char *buf = malloc(1048576);
    memset(buf, 0, 1048576);

    for (int k = 0; k < nsizes; k++) {
        int n = sizes[k];
        MPI_Barrier(MPI_COMM_WORLD);
        double t0 = MPI_Wtime();

        for (int i = 0; i < ITER; i++) {
            if (rank == 0) {
                MPI_Send(buf, n, MPI_CHAR, 1, 0, MPI_COMM_WORLD);                 /* ping */
                MPI_Recv(buf, n, MPI_CHAR, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE); /* pong */
            } else {
                MPI_Recv(buf, n, MPI_CHAR, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                MPI_Send(buf, n, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
            }
        }

        double t1 = MPI_Wtime();
        if (rank == 0) {
            double rtt = (t1 - t0) / ITER;          /* average round-trip time */
            double one_way = rtt / 2.0;             /* estimate of one-way time */
            double bw = n / one_way / 1.0e6;        /* MB/s */
            printf("Message size : %d bytes\n", n);
            printf("Iterations : %d\n", ITER);
            printf("Total round trips : %d\n", ITER);
            printf("Average round-trip : %.9f seconds\n", rtt);
            printf("Average one-way : %.9f seconds\n", one_way);
            printf("Bandwidth : %.2f MB/s\n\n", bw);
```

```
        }
    }

    free(buf);
    MPI_Finalize();
    return 0;
}
```

3. Step-by-step explanation (for the lab exam)

- **Ping-pong:** rank 0 sends the message (ping), rank 1 receives it and sends it straight back (pong). One ping + one pong = one round trip.
- **Why rank 0 sends first and rank 1 receives first:** opposite orders on the two sides guarantee each Send has a matching Recv already (or soon) waiting, so there is no deadlock.
- **Average round-trip** = total elapsed time / number of iterations. **One-way time** = round-trip / 2 (an estimate; we assume both directions cost the same).
- **Bandwidth** = message size / one-way time. Printed in MB/s by dividing bytes by 10^6.
- **Latency** is the time for a very small message (64 bytes here): it is almost all start-up cost, not data movement. **Bandwidth** is best seen with large messages (1 MB) where start-up cost is negligible.
- **Analysis to write in the report:** plot one-way time vs size. The y-intercept is latency; the slope is 1/bandwidth. Small sizes stay flat, then time grows linearly. You may see a jump around 16-64 KB where MPI switches from the *eager* protocol (copies message into a buffer) to the *rendezvous* protocol (handshake before sending).
- **Accuracy tips:** 5 iterations is only enough to match the sample format. For real numbers use hundreds or thousands of iterations, discard the first (warm-up) iteration, and repeat the whole experiment.

4. MPI calls used

- `MPI_Send / MPI_Recv` with `MPI_CHAR` so count = number of bytes.
- `MPI_Wtime` for timing, `MPI_Barrier` so both ranks start together.

5. Viva questions and model answers

Q. Define latency and bandwidth.

Latency = fixed time to start and deliver a message of (almost) zero size. Bandwidth = rate at which data is transferred once the transfer is under way, bytes per second. Simple model: T = latency + size / bandwidth.

Q. Why divide the round-trip by 2?

A round trip contains two equal transfers (there and back), so one-way is estimated as half. Only rank 0's clock is needed, which avoids clock-synchronisation problems between processes.

Q. Why not measure one-way time directly?

It would need synchronised clocks on both processes. Clocks on different nodes differ slightly, so the round-trip method is more reliable.

Q. Why does bandwidth look poor for small messages?

Latency dominates the time, so size / time is small even though the network is fast. Bandwidth approaches the hardware limit only for large messages.

Q. What are eager and rendezvous protocols?

Eager: the sender copies small messages to a buffer and returns quickly. Rendezvous: for large messages the sender and receiver first handshake, then data is transferred directly. This explains a change in the time curve at a certain message size.

Q. Why is the number of iterations important?

Single measurements are noisy. Averaging removes timer resolution problems and run-to-run variation.

Q. Where do the two processes run when you test on one laptop?

On the same machine, so MPI uses shared memory, which is much faster than a network. For network latency/bandwidth you need processes on different nodes.

6. Common mistakes (examiners look for these)

- Using MPI_INT with a byte count, so the actual bytes sent are 4x bigger than reported.

- Timing only one iteration or including the printf inside the timed region.
- Both ranks sending first (deadlock risk for large messages).
- Buffer smaller than the largest message size (buffer overflow).

Problem 5: Distributed Matrix Multiplication

1. Question (from the assignment)

Distribute matrix rows from the root process to other processes using MPI_Send. Each process performs its portion of matrix multiplication and sends the result back to the root.

Sample Input

```
Number of processes: 2
Matrix A:
1 2
3 4
Matrix B:
5 6
7 8
```

Sample Output

```
Matrix C = A x B
19 22
43 50
C[0][0] = 1x5 + 2x7 = 19
C[0][1] = 1x6 + 2x8 = 22
C[1][0] = 3x5 + 4x7 = 43
C[1][1] = 3x6 + 4x8 = 50
```

2. Solution code

Compile / run: `mpicc p5.c -o p5 && mpirun -np 2 ./p5`

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2) {                    /* one row of A per process, 2x2 matrices */
        if (rank == 0) printf("Run with exactly 2 processes.\n");
        MPI_Finalize();
        return 1;
    }

    int A[2][2], B[2][2], C[2][2];
    int local_A[2], local_C[2] = {0, 0};

    if (rank == 0) {
        int a[2][2] = {{1, 2}, {3, 4}};
        int b[2][2] = {{5, 6}, {7, 8}};
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 2; j++) { A[i][j] = a[i][j]; B[i][j] = b[i][j]; }
    }

    /* every process needs ALL of B, but only ONE row of A */
    MPI_Bcast(B, 4, MPI_INT, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        local_A[0] = A[0][0];
        local_A[1] = A[0][1];
        MPI_Send(A[1], 2, MPI_INT, 1, 0, MPI_COMM_WORLD);      /* row 1 -> P1 */
    } else {
        MPI_Recv(local_A, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }

    /* local_C[j] = sum over k of local_A[k] * B[k][j]  (one row of C) */
    for (int j = 0; j < 2; j++)
        for (int k = 0; k < 2; k++)
            local_C[j] += local_A[k] * B[k][j];

    if (rank == 0) {
        C[0][0] = local_C[0];
        C[0][1] = local_C[1];
        MPI_Recv(C[1], 2, MPI_INT, 1, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Matrix C = A x B\n");
```

```
        printf("%d %d\n%d %d\n", C[0][0], C[0][1], C[1][0], C[1][1]);
        printf("C[0][0] = %d\nC[0][1] = %d\nC[1][0] = %d\nC[1][1] = %d\n",
               C[0][0], C[0][1], C[1][0], C[1][1]);
    } else {
        MPI_Send(local_C, 2, MPI_INT, 0, 1, MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}
```

3. Step-by-step explanation (for the lab exam)

- **Row decomposition:** row i of C = (row i of A) x B. So if process i owns row i of A and a full copy of B, it can compute row i of C alone, with no communication during the computation.
- **Why broadcast B:** every process needs the whole of B. `MPI_Bcast` sends it from root to all. (If you want only MPI_Send, root can send B to each worker too; Bcast is the standard way.)
- **Distribute A:** root keeps row 0 and sends row 1 using `MPI_Send`; process 1 receives it into `local_A`.
- **Local computation:** `local_C[j] += local_A[k] * B[k][j]` is the dot product of the local row with column j of B. For P0: [1 2] x B = [1x5+2x7, 1x6+2x8] = [19 22]. For P1: [3 4] x B = [43 50].
- **Collect:** P1 sends its result row (tag 1); root receives it directly into `C[1]`, so the final matrix is assembled in order.
- **Generalising:** for an N x N matrix and P processes, give each process N/P rows (use MPI_Scatter or Scatterv), Bcast B, compute N/P rows, and MPI_Gather the result.

4. MPI calls used

- `MPI_Send / MPI_Recv` - distribute a row of A and return a row of C.
- `MPI_Bcast` - give every process the full matrix B.

5. Viva questions and model answers

Q. Why does each process need all of B but only one row of A?

Row i of C is the dot product of row i of A with every column of B, so the whole of B is required but only that one row of A.

Q. Write the formula for matrix multiplication.

C[i][j] = sum over k of A[i][k] x B[k][j].

Q. Is the loop local_C[j] += local_A[k] * B[k][j] correct?

Yes. It computes one row of C. (The loop in the supplied solution file, local_C[i] += local_A[j] * B[j][i], is the same formula with the letters renamed, so it is also correct. The note in that file saying it was wrong is mistaken.)

Q. What is the computational cost and how does parallelism help?

Serial cost is O(N^3). With P processes each handles N/P rows, so computation per process is O(N^3 / P), but we pay extra for communication (sending A rows, broadcasting B, gathering C).

Q. What is the communication overhead in this approach?

Broadcasting B costs N^2 elements to every process, which is the largest term. Distributing A and collecting C is only N^2 total.

Q. How would you handle N not divisible by P?

Use MPI_Scatterv and MPI_Gatherv with counts and displacements (like Problem 13 of the assignment).

Q. Why is C[1] received directly into the matrix?

A row of a C array is contiguous in memory, so receiving 2 ints into C[1] fills the entire row.

6. Common mistakes (examiners look for these)

- Multiplying element by element (A[i][j] x B[i][j]) instead of row by column.
- Forgetting to give the workers matrix B.
- Not initialising local_C to 0 before using +=.
- Running with a different number of processes than rows.

Problem 7: Nonblocking Communication (MPI_Isend / MPI_Irecv)

1. Question (from the assignment)

Implement a distributed vector computation using MPI_Isend and MPI_Irecv. Compare its execution time with the equivalent blocking MPI_Send/MPI_Recv implementation.

Sample Input

```
Number of processes: 2
Array size = 8
P0: 10 20 30 40 50 60 70 80
P1: 1 2 3 4 5 6 7 8
Each process exchanges its array using MPI_Isend and MPI_Irecv.
```

Sample Output

```
P0 received: 1 2 3 4 5 6 7 8
P1 received: 10 20 30 40 50 60 70 80
Nonblocking communication completed.
Sample performance:
Blocking communication time = 0.000021 s
Nonblocking communication time = 0.000015 s
```

2. Solution code

Compile / run: `mpicc p7.c -o p7 && mpirun -np 2 ./p7`

```
#include <mpi.h>
#include <stdio.h>

#define N 8
#define REPS 1000

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2) {
        if (rank == 0) printf("Run with exactly 2 processes.\n");
        MPI_Finalize();
        return 1;
    }

    int send_arr[N], recv_arr[N];
    for (int i = 0; i < N; i++) send_arr[i] = (rank == 0) ? (i + 1) * 10 : (i + 1);
    int other = 1 - rank;
    MPI_Request reqs[2];

    /* ---------------- blocking exchange (ordered to avoid deadlock) ---------------- */
    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = MPI_Wtime();
    for (int r = 0; r < REPS; r++) {
        if (rank == 0) {
            MPI_Send(send_arr, N, MPI_INT, other, 0, MPI_COMM_WORLD);
            MPI_Recv(recv_arr, N, MPI_INT, other, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        } else {
            MPI_Recv(recv_arr, N, MPI_INT, other, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            MPI_Send(send_arr, N, MPI_INT, other, 0, MPI_COMM_WORLD);
        }
    }
    double t_block = (MPI_Wtime() - t0) / REPS;

    /* ---------------- nonblocking exchange ---------------- */
    MPI_Barrier(MPI_COMM_WORLD);
    t0 = MPI_Wtime();
    for (int r = 0; r < REPS; r++) {
        MPI_Isend(send_arr, N, MPI_INT, other, 1, MPI_COMM_WORLD, &reqs[0]);
        MPI_Irecv(recv_arr, N, MPI_INT, other, 1, MPI_COMM_WORLD, &reqs[1]);
        MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);     /* buffers are safe only after this */
    }
    double t_nb = (MPI_Wtime() - t0) / REPS;
```

```
    printf("P%d received:", rank);
    for (int i = 0; i < N; i++) printf(" %d", recv_arr[i]);
    printf("\n");
    fflush(stdout);
    MPI_Barrier(MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Nonblocking communication completed.\n");
        printf("Blocking communication time    = %.9f s\n", t_block);
        printf("Nonblocking communication time = %.9f s\n", t_nb);
    }

    MPI_Finalize();
    return 0;
}
```

3. Step-by-step explanation (for the lab exam)

- **Blocking exchange:** `MPI_Send`/`MPI_Recv` return only when the buffer can be reused / the data has arrived. To exchange safely, rank 0 sends then receives and rank 1 receives then sends (opposite order).
- **Nonblocking exchange:** `MPI_Isend` and `MPI_Irecv` only *start* the transfer and return immediately with a request handle. Both ranks can post send and receive in the same order without deadlock, because nobody waits at the call.
- **MPI_Waitall:** waits until both requests complete. You must not read `recv_arr` or modify `send_arr` before this point.
- **Timing:** each version is run 1000 times and averaged. A barrier makes both ranks start together.
- **Interpreting the result:** nonblocking is not automatically faster. For 8 integers the two are close. Its real benefit is *overlap*: do useful computation between the Isend/Irecv and the Wait while the data is in flight. Here there is no computation to overlap, so any difference is small.
- **Distinct tags** (0 for blocking, 1 for nonblocking) keep the two phases from matching each other's messages.

4. MPI calls used

- `MPI_Isend(buf, count, type, dest, tag, comm, &request)` - start nonblocking send.
- `MPI_Irecv(buf, count, type, source, tag, comm, &request)` - start nonblocking receive.
- `MPI_Waitall(count, requests, statuses)` - wait for all requests to finish.
- `MPI_Wtime, MPI_Barrier` - timing.

5. Viva questions and model answers

Q. Difference between blocking and nonblocking communication?

A blocking call returns only when its buffer is safe to reuse (send) or the data has arrived (receive). A nonblocking call returns immediately; completion must be checked with MPI_Wait/MPI_Test.

Q. Why can both processes safely call Isend then Irecv?

Isend does not wait for a matching receive, so neither process is stuck before posting its receive. That removes the deadlock that two blocking sends could cause.

Q. What is an MPI_Request?

A handle for a pending nonblocking operation. It is passed to MPI_Wait, MPI_Waitall or MPI_Test to complete or check the operation.

Q. What is wrong with reading recv_arr right after MPI_Irecv?

The data may not have arrived yet. You must call MPI_Wait/Waitall first. Also do not modify the send buffer until the send completes.

Q. When does nonblocking communication give a real speed-up?

When you can overlap computation with communication (for example, compute on interior points while halo values are in transit). With no independent work, there is little or no benefit.

Q. What is the difference between MPI_Wait and MPI_Test?

MPI_Wait blocks until the operation completes. MPI_Test returns immediately with a flag saying whether it has completed.

Q. Does Isend guarantee that the message has been received?

No. Completion of an Isend only means the send buffer can be reused, not that the receiver has the data.

6. Common mistakes (examiners look for these)

- Using the buffers before MPI_Wait / MPI_Waitall.
- Forgetting to wait, so the program may finish with pending operations.
- Using one MPI_Request for two operations at the same time.
- Claiming nonblocking is always faster without any overlapping work.

Problem 8: Avoiding Deadlock

1. Question (from the assignment)

Create a two-process communication program that can cause deadlock when blocking sends are used. Modify the program to avoid deadlock using appropriate MPI communication techniques.

Sample Input

```
Number of processes: 2
P0 message = 100
P1 message = 200
```

Sample Output

```
Unsafe version:
P0: Waiting to send 100
P1: Waiting to send 200
Potential deadlock detected.

Corrected version:
P0 sent 100
P1 received 100
P1 sent 200
P0 received 200
Communication completed without deadlock.
```

2. Solution code

Compile / run: Unsafe (will hang - stop with Ctrl+C): `mpirun -np 2 ./p8 unsafe` Safe: `mpirun -np 2 ./p8`

```
#include <mpi.h>
#include <stdio.h>
#include <string.h>

#define N 1000000                 /* 4 MB: too big to be buffered internally */
static int sendbuf[N], recvbuf[N];

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2) {
        if (rank == 0) printf("Run with exactly 2 processes.\n");
        MPI_Finalize();
        return 1;
    }

    int msg = (rank == 0) ? 100 : 200;
    int other = 1 - rank;
    for (int i = 0; i < N; i++) sendbuf[i] = msg;

    int unsafe = (argc > 1 && strcmp(argv[1], "unsafe") == 0);

    if (unsafe) {
        /* UNSAFE: both processes send first, then receive */
        printf("P%d: Waiting to send %d\n", rank, msg);
        fflush(stdout);
        MPI_Send(sendbuf, N, MPI_INT, other, 0, MPI_COMM_WORLD);   /* both block here */
        MPI_Recv(recvbuf, N, MPI_INT, other, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("P%d: not reached if deadlock occurs\n", rank);
    } else {
        /* SAFE: one call does the send and the receive together */
        MPI_Sendrecv(sendbuf, N, MPI_INT, other, 0,
                     recvbuf, N, MPI_INT, other, 0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("P%d sent %d, received %d\n", rank, msg, recvbuf[0]);
    }

    MPI_Barrier(MPI_COMM_WORLD);
    if (rank == 0 && !unsafe) printf("Communication completed without deadlock.\n");

    MPI_Finalize();
```

```
    return 0;
}
```

Other correct ways to fix the deadlock (replace the unsafe block with one of these):

```
/* Fix 2: break the symmetry - opposite order on the two ranks */
if (rank == 0) {
    MPI_Send(sendbuf, N, MPI_INT, 1, 0, MPI_COMM_WORLD);
    MPI_Recv(recvbuf, N, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
} else {
    MPI_Recv(recvbuf, N, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    MPI_Send(sendbuf, N, MPI_INT, 0, 0, MPI_COMM_WORLD);
}
```

```
/* Fix 3: nonblocking calls */
MPI_Request reqs[2];
MPI_Irecv(recvbuf, N, MPI_INT, other, 0, MPI_COMM_WORLD, &reqs[0]);
MPI_Isend(sendbuf, N, MPI_INT, other, 0, MPI_COMM_WORLD, &reqs[1]);
MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);
```

3. Step-by-step explanation (for the lab exam)

- **Why the unsafe version deadlocks:** both processes call `MPI_Send` first. For a large message MPI uses the rendezvous protocol: the send cannot finish until the other side posts a matching receive. But each process is stuck inside its own send, so neither ever reaches its receive. This is a circular wait = deadlock.
- **Why a big buffer is used:** for tiny messages (like a single int) MPI usually copies the data into an internal buffer and `MPI_Send` returns, so the unsafe code may appear to work. That is *unsafe* programming - correctness depends on buffering, which the MPI standard does not guarantee. A 4 MB message forces the blocking behaviour so the deadlock is reproducible.
- **Fix 1 - MPI_Sendrecv:** performs the send and the receive in one call, and MPI schedules them internally so they cannot block each other. It is the shortest and safest fix for exchange patterns.
- **Fix 2 - reorder:** rank 0 sends then receives; rank 1 receives then sends. The circular dependency is broken.
- **Fix 3 - nonblocking:** post the receive and send without waiting, then wait for both.
- **Output:** safe version prints 'P0 sent 100, received 200' and 'P1 sent 200, received 100', followed by the completion message.

4. MPI calls used

- `MPI_Sendrecv(sendbuf, scount, stype, dest, stag, recvbuf, rcount, rtype, source, rtag, comm, status)`
- `MPI_Send / MPI_Recv` (unsafe pattern), `MPI_Isend / MPI_Irecv / MPI_Waitall` (alternative fix).

5. Viva questions and model answers

Q. What is deadlock?

A situation where processes wait forever for events that can only be caused by each other - here each process waits in MPI_Send for a receive that the other process can never post.

Q. Why does the unsafe code sometimes work for small messages?

MPI may copy small messages into an internal buffer (eager protocol), so MPI_Send returns without a matching receive. Relying on that is unsafe because buffer sizes are implementation-dependent.

Q. Name three ways to avoid this deadlock.

MPI_Sendrecv; reorder sends and receives (even ranks send first, odd ranks receive first); nonblocking Isend/Irecv with Waitall. Also MPI_Bsend with an attached buffer.

Q. How does MPI_Sendrecv avoid deadlock?

The send and receive are progressed together inside one call, so the process is always able to receive while its send is pending.

Q. What is the difference between MPI_Send, MPI_Ssend and MPI_Bsend?

Standard send may or may not buffer. Ssend completes only when the receive has started (synchronous). Bsend copies to a user-attached buffer and always returns quickly.

Q. Does the MPI standard say MPI_Send is blocking?

It says the call may return only when the send buffer can be reused, and it may or may not wait for the receiver. Programs must be correct even if it waits (the 'unsafe program' definition).

6. Common mistakes (examiners look for these)

- Demonstrating the deadlock with a single small int and being surprised it does not hang.
- Using the same send-first order on both ranks and calling it fixed.
- Mismatched tags or counts in Sendrecv.
- Forgetting MPI_Waitall after Isend/Irecv.

Problem 10: Producer-Consumer System

1. Question (from the assignment)

Design a producer-consumer system where one or more MPI processes generate data and another process receives and processes the data using point-to-point communication.

Sample Input

```
Number of processes: 3
P0 = Producer
P1 = Consumer
P2 = Consumer
Number of items = 8
Items: 10 20 30 40 50 60 70 80
```

Sample Output

```
P0 produced: 10
P1 consumed: 10
P0 produced: 20
P2 consumed: 20
P0 produced: 30
P1 consumed: 30
P0 produced: 40
P2 consumed: 40
...
All 8 items processed successfully.
```

2. Solution code

Compile / run: `mpicc p10.c -o p10 && mpirun -np 3 ./p10` (any number of processes from 2 up works)

```
#include <mpi.h>
#include <stdio.h>

#define NITEMS 8

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        if (rank == 0) printf("Need at least 2 processes (1 producer + consumers).\n");
        MPI_Finalize();
        return 1;
    }

    int items[NITEMS] = {10, 20, 30, 40, 50, 60, 70, 80};
    int consumers = size - 1;                /* ranks 1 .. size-1 */

    if (rank == 0) {                         /* PRODUCER */
        for (int i = 0; i < NITEMS; i++) {
            int target = (i % consumers) + 1; /* round-robin: 1,2,1,2,... */
            printf("P0 produced: %d\n", items[i]);
            MPI_Send(&items[i], 1, MPI_INT, target, 0, MPI_COMM_WORLD);
        }
    } else {                                 /* CONSUMER */
        int my_count = 0;                    /* how many items are sent to me */
        for (int i = rank - 1; i < NITEMS; i += consumers) my_count++;

        for (int k = 0; k < my_count; k++) {
            int item;
            MPI_Recv(&item, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            printf("P%d consumed: %d\n", rank, item);
        }
    }

    MPI_Barrier(MPI_COMM_WORLD);             /* wait until everyone is done */
    if (rank == 0) printf("All %d items processed successfully.\n", NITEMS);

    MPI_Finalize();
    return 0;
```

```
}
```

3. Step-by-step explanation (for the lab exam)

- **Roles by rank:** rank 0 is the producer; every other rank is a consumer. Role assignment by rank is the standard SPMD technique.
- **Dispatch rule:** item i goes to consumer `(i % consumers) + 1`. With 2 consumers: items 0,2,4,6 -> P1 and items 1,3,5,7 -> P2 (10,30,50,70 to P1 and 20,40,60,80 to P2). This is static round-robin.
- **Consumer loop:** each consumer must know exactly how many receives to post. The loop `for (i = rank-1; i < NITEMS; i += consumers)` counts the items that will be sent to it. This works for any number of consumers (the original solution hard-coded 4 receives).
- **Barrier + final message:** the barrier guarantees every consumer has finished before the producer prints 'All 8 items processed successfully.'
- **Output order:** 'produced' and 'consumed' lines come from different processes, so their interleaving on screen is not fixed and may not match the sample exactly. The sample shows one possible interleaving.
- **Extension (dynamic version, often asked in viva):** consumers send a 'ready' message; the producer receives it with `MPI_ANY_SOURCE`, reads `status.MPI_SOURCE` and sends the next item to that consumer. A special tag (or value) tells consumers to stop. This balances load when processing times differ.

4. MPI calls used

- `MPI_Send / MPI_Recv` - item transfer producer -> consumer.
- `MPI_Barrier` - synchronise before the final message.
- (Dynamic version) `MPI_ANY_SOURCE, MPI_Status.MPI_SOURCE, MPI_Status.MPI_TAG`.

5. Viva questions and model answers

Q. What is the producer-consumer model?

One or more producers generate data and one or more consumers process it. It is a master-worker style of work distribution.

Q. How is the work distributed in your program?

Static round-robin: item i goes to consumer (i mod number_of_consumers)+1.

Q. What is the drawback of static distribution?

If items take different processing times, some consumers finish early and idle while others are overloaded. Dynamic assignment (consumer asks for work) balances load.

Q. What is MPI_ANY_SOURCE?

A wildcard for the source argument of MPI_Recv so a message from any process matches. The actual sender is found in status.MPI_SOURCE.

Q. How does a consumer know when to stop in a dynamic design?

The producer sends a termination message, for example with a special tag or a sentinel value like -1.

Q. Why does the consumer need to know how many items to expect here?

Because there is no termination message; each consumer posts exactly as many receives as items it will get. Posting extra receives would hang forever.

Q. Why can the printed output look different from the sample?

Processes run concurrently and write to the screen independently, so line order is non-deterministic.

6. Common mistakes (examiners look for these)

- Hard-coding 4 receives per consumer; works only for 3 processes and 8 items.
- A consumer posting more receives than items it receives - it hangs.
- Sending to rank 0 (the producer) or to rank = size (does not exist).
- Expecting the exact line order of the sample output.

Problem 11: Broadcast of Configuration Parameters

1. Question (from the assignment)

The root process reads configuration parameters and broadcasts them to all MPI processes. Each process uses the received parameters to perform a computation.

Sample Input

```
Number of processes: 4
Root process reads:
Number of iterations = 1000
Threshold = 50
Block size = 256
```

Sample Output

```
P0 received: Iterations = 1000, Threshold = 50, Block size = 256
P1 received: Iterations = 1000, Threshold = 50, Block size = 256
P2 received: Iterations = 1000, Threshold = 50, Block size = 256
P3 received: Iterations = 1000, Threshold = 50, Block size = 256
```

2. Solution code

Compile / run: `mpicc p11.c -o p11 && mpirun -np 4 ./p11` (works for any number of processes)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int config[3];                /* [0]=iterations, [1]=threshold, [2]=block size */

    if (rank == 0) {              /* only the root "reads" the parameters */
        config[0] = 1000;
        config[1] = 50;
        config[2] = 256;
        /* To read from keyboard instead:
           scanf("%d %d %d", &config[0], &config[1], &config[2]); */
    }

    /* ALL processes must call Bcast. Root sends, others receive into the same array. */
    MPI_Bcast(config, 3, MPI_INT, 0, MPI_COMM_WORLD);

    int iterations = config[0], threshold = config[1], block = config[2];
    printf("P%d received: Iterations = %d, Threshold = %d, Block size = %d\n",
           rank, iterations, threshold, block);

    /* a small computation that uses the parameters and depends on the rank */
    int count = 0;
    for (int i = 0; i < iterations; i++)
        if ((i * (rank + 1)) % 100 < threshold) count++;
    int blocks = (iterations + block - 1) / block;        /* ceiling division */
    printf("P%d computed: %d of %d iterations below threshold, %d blocks of size %d\n",
           rank, count, iterations, blocks, block);

    MPI_Finalize();
    return 0;
}
```

3. Step-by-step explanation (for the lab exam)

- **Why broadcast:** only the root knows the parameters. `MPI_Bcast` copies the same data from the root to every process in the communicator in one collective call.
- **The same call on every rank:** `MPI_Bcast(config, 3, MPI_INT, 0, MPI_COMM_WORLD)` is executed by *all* processes. On the root it is the sender; on the others `config` is the receive buffer. The root argument (0) must be identical everywhere.
- **No tag and no matching Recv:** collective operations do not need tags or separate receive calls. All processes in the communicator must participate, otherwise the program hangs.

- **Use of the parameters:** each rank loops *iterations* times and counts how many values satisfy the threshold condition, then computes the number of blocks of size *block* needed. The computation differs per rank because it uses `rank + 1`, which proves each process is really using the broadcast data.
- **Reading input:** in real programs only rank 0 calls scanf (or reads a file), then broadcasts. Doing scanf on all ranks is unreliable because stdin is normally connected only to rank 0.
- **Performance idea:** MPI implementations broadcast using a tree, so the cost is about O(log P) steps rather than P-1 separate sends.

4. MPI calls used

- `MPI_Bcast(buffer, count, datatype, root, comm)` - one-to-all collective.

5. Viva questions and model answers

Q. What is a collective operation?

A communication operation in which all processes of a communicator participate (Bcast, Scatter, Gather, Reduce, ...). Every process must call it.

Q. Who calls MPI_Bcast?

Every process in the communicator, with the same root, count and datatype.

Q. Difference between MPI_Bcast and MPI_Scatter?

Bcast sends the *same* whole data to everyone. Scatter splits the root's data into different pieces, one per process.

Q. What happens if only the root calls MPI_Bcast?

The program hangs or behaves incorrectly because other processes never participate.

Q. Why is MPI_Bcast better than a loop of MPI_Send?

It is simpler and implementations use tree algorithms, so it takes about log2(P) steps instead of P-1 sequential sends from the root.

Q. Does MPI_Bcast synchronise all processes?

Not guaranteed. The standard does not require a barrier effect, although a process cannot get the data before the root has called it.

Q. Can you broadcast a structure or a string?

Yes: a string as MPI_CHAR with its length, a struct by creating an MPI derived datatype or by packing.

6. Common mistakes (examiners look for these)

- Calling MPI_Bcast only inside `if (rank == 0)`.
- Using different root values on different ranks.
- Using the wrong count (broadcasting 1 value instead of 3).
- Calling scanf on all ranks.

Problem 15: Trapezoidal Rule Integration

1. Question (from the assignment)

Implement the trapezoidal rule for numerical integration. Each MPI process calculates a portion of the integral, and MPI_Reduce combines the partial results. Students should report partial integrals and repeat the experiment with different numbers of intervals.

Sample Input

```
Number of processes: 4
Function: f(x) = x^2
Lower limit = 0
Upper limit = 1
Number of intervals = 1000
Use the trapezoidal rule.
```

Sample Output

```
Numerical integral = 0.333333
Exact integral = 0.333333
Absolute error = 0.000000
```

2. Solution code

Compile / run: `mpicc p15.c -o p15 -lm && mpirun -np 4 ./p15` (`-lm` is needed for fabs)

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

double f(double x) { return x * x; }

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    double a = 0.0, b = 1.0;
    double exact = 1.0 / 3.0;
    int ns[] = {10, 100, 1000, 10000, 100000};       /* different numbers of intervals */
    int nn = sizeof(ns) / sizeof(ns[0]);

    for (int k = 0; k < nn; k++) {
        int n = ns[k];
        double h = (b - a) / n;

        /* split n intervals over size processes; first 'rem' ranks get one extra */
        int base = n / size, rem = n % size;
        int local_n = base + (rank < rem ? 1 : 0);
        int start   = rank * base + (rank < rem ? rank : rem);

        double local_a = a + start * h;
        double local_b = local_a + local_n * h;
        double local_int = 0.0;
        if (local_n > 0) {
            local_int = (f(local_a) + f(local_b)) / 2.0;   /* end points: half weight */
            for (int i = 1; i < local_n; i++)
                local_int += f(local_a + i * h);           /* interior points: full weight */
            local_int *= h;
        }

        double total = 0.0;
        MPI_Reduce(&local_int, &total, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

        double *parts = NULL;
        if (rank == 0) parts = malloc(size * sizeof(double));
        MPI_Gather(&local_int, 1, MPI_DOUBLE, parts, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);

        if (rank == 0) {
            printf("---- n = %d intervals ----\n", n);
            for (int r = 0; r < size; r++)
                printf("  Partial integral P%d = %.8f\n", r, parts[r]);
            printf("Numerical integral = %.8f\n", total);
```

```
            printf("Exact integral     = %.8f\n", exact);
            printf("Absolute error     = %.3e\n\n", fabs(total - exact));
            free(parts);
        }
    }

    MPI_Finalize();
    return 0;
}
```

3. Step-by-step explanation (for the lab exam)

- **Trapezoidal rule:** split [a,b] into n pieces of width h = (b-a)/n. Area = h x [f(x0)/2 + f(x1) + ... + f(x(n-1)) + f(xn)/2]. Each trapezoid area is h x (f(left) + f(right)) / 2; adding them makes interior points count twice (full weight) and the two end points once (half weight).
- **Parallelisation:** the integral is a sum over sub-intervals, and each sub-interval is independent. Each process integrates its own sub-range [local_a, local_b] with its own local_n trapezoids using the same h.
- **Dividing the work:** base = n / size, rem = n % size; the first 'rem' ranks take one extra interval, and 'start' is the index where a rank's block begins. This handles n not divisible by the process count (the simple n/size version silently drops intervals).
- **Combine:** `MPI_Reduce(..., MPI_SUM, 0, ...)` adds all partial integrals into `total` on rank 0. `MPI_Gather` collects the individual partial integrals so they can be reported, as the assignment asks.
- **Hand check with 4 processes, n = 1000:** each process gets 250 intervals, i.e. a quarter of [0,1]. The four partial integrals are approximately 1/192, 7/192, 19/192, 37/192 (about 0.0052, 0.0365, 0.0990, 0.1927) and sum to 1/3.
- **Accuracy:** trapezoid error for f(x)=x^2 on [0,1] is h^2 / 6. For n = 1000, h = 0.001 so the numerical value is about 0.33333350 and the absolute error about 1.7e-7. It prints as 0.333333 with six decimals, as in the sample. Increasing n by 10 reduces the error by about 100 (second-order accurate).

4. MPI calls used

- `MPI_Reduce(sendbuf, recvbuf, count, type, MPI_SUM, root, comm)` - combine partial results at root.
- `MPI_Gather` - collect one partial value per rank for reporting.
- `MPI_Comm_size / MPI_Comm_rank` - to divide the interval.

5. Viva questions and model answers

Q. State the trapezoidal rule.

Integral of f from a to b is approximately h x [(f(a) + f(b)) / 2 + sum of f(x_i) for i = 1 .. n-1], with h = (b - a) / n.

Q. Why is this problem easy to parallelise?

The sub-interval integrals are independent, so there is no communication until the final sum. It is an 'embarrassingly parallel' problem with a reduction at the end.

Q. What does MPI_Reduce do?

Combines one value from every process with an operation (MPI_SUM here) and puts the result on the root process only.

Q. Difference between MPI_Reduce and MPI_Allreduce?

Reduce delivers the result to one root only; Allreduce delivers it to every process.

Q. What is the exact value and why is the numerical value slightly larger?

Exact = 1/3. The function x^2 is convex, so the straight top edges of trapezoids lie above the curve, giving a slight overestimate (by about h^2/6).

Q. What happens if n is not divisible by the number of processes?

With plain n/size, leftover intervals are lost and the answer is wrong. This program gives the remainder intervals to the first ranks.

Q. How does the error change with n?

It decreases as O(h^2) = O(1/n^2): ten times more intervals gives about 100 times smaller error.

Q. Why does each process need the same h?

So all trapezoids have equal width and the sub-ranges line up exactly without gaps or overlap.

6. Common mistakes (examiners look for these)

- Computing local_a with the wrong offset so ranges overlap or leave gaps.
- Dropping the remainder intervals when n % size != 0.
- Giving end points of every sub-range full weight, or interior points half weight.
- Forgetting -lm when using fabs.
- Printing the result on every rank instead of only root.

Problem 16: Prefix Sum with MPI_Scan

1. Question (from the assignment)

Each MPI process stores a local value. Use MPI_Scan to calculate the prefix sum across all processes and compare it with a manually implemented point-to-point solution.

Sample Input

```
Number of processes: 4
P0 = 10
P1 = 20
P2 = 30
P3 = 40
```

Sample Output

```
P0: Local value = 10, Prefix sum = 10
P1: Local value = 20, Prefix sum = 30
P2: Local value = 30, Prefix sum = 60
P3: Local value = 40, Prefix sum = 100
Prefix result: 10 30 60 100
```

2. Solution code

Compile / run: `mpicc p16.c -o p16 && mpirun -np 4 ./p16` (works for any process count)

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int local_val = (rank + 1) * 10;              /* 10, 20, 30, 40, ... */

    /* ---- Method 1: library collective ---- */
    int scan_result;
    MPI_Scan(&local_val, &scan_result, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);

    /* ---- Method 2: manual point-to-point (a chain) ---- */
    int manual_result = local_val;
    if (rank > 0) {
        int left_sum;                              /* prefix sum of ranks 0..rank-1 */
        MPI_Recv(&left_sum, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        manual_result += left_sum;
    }
    if (rank < size - 1)
        MPI_Send(&manual_result, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD);

    printf("P%d: Local value = %d, Prefix sum = %d (MPI_Scan), %d (manual) -> %s\n",
           rank, local_val, scan_result, manual_result,
           scan_result == manual_result ? "match" : "MISMATCH");
    fflush(stdout);
    MPI_Barrier(MPI_COMM_WORLD);

    /* collect the prefix sums on root to print them as one line */
    int *all = NULL;
    if (rank == 0) all = malloc(size * sizeof(int));
    MPI_Gather(&scan_result, 1, MPI_INT, all, 1, MPI_INT, 0, MPI_COMM_WORLD);
    if (rank == 0) {
        printf("Prefix result:");
        for (int i = 0; i < size; i++) printf(" %d", all[i]);
        printf("\n");
        free(all);
    }

    MPI_Finalize();
    return 0;
}
```

3. Step-by-step explanation (for the lab exam)

- **Prefix (scan) sum:** process r receives the sum of the values of ranks 0 .. r. With values 10, 20, 30, 40 the results are 10, 30, 60, 100.
- **MPI_Scan:** every process calls `MPI_Scan(&local_val, &result, 1, MPI_INT, MPI_SUM, comm)`. It is *inclusive*: rank r's own value is included. (MPI_Exscan is exclusive and leaves rank 0's result undefined.)
- **Manual version (a pipeline):** rank 0 starts with its own value and sends it to rank 1. Rank 1 receives the prefix from rank 0, adds its own value, and sends to rank 2, and so on. The last rank only receives. The receive is skipped on rank 0 and the send is skipped on the last rank.
- **Why no deadlock:** rank 0 never waits for anyone, so it can always send; each next rank is unblocked by the previous one. The chain is acyclic.
- **Comparison:** both give identical results. The manual chain takes P-1 sequential steps (O(P) time); library implementations use tree-like algorithms and finish in about O(log P) steps, so MPI_Scan scales better for many processes. For 4 processes the difference is negligible.
- **Gather at the end** only collects the scan results at rank 0 so one 'Prefix result' line is printed in rank order.

4. MPI calls used

- `MPI_Scan(sendbuf, recvbuf, count, type, op, comm)` - inclusive prefix reduction.
- `MPI_Send / MPI_Recv` - manual chain. `MPI_Gather` - collect results for printing.

5. Viva questions and model answers

Q. What is a prefix sum?

For values x0..x(P-1), prefix sum at i = x0 + x1 + ... + xi.

Q. Inclusive vs exclusive scan?

Inclusive (MPI_Scan) includes the process's own value. Exclusive (MPI_Exscan) gives the sum of values of lower ranks only; rank 0's output is undefined.

Q. Difference between MPI_Scan and MPI_Reduce?

Reduce gives one total at the root. Scan gives every process a partial (prefix) total.

Q. Why is your manual version sequential?

Rank r cannot compute its result until it receives the prefix from rank r-1, so the steps happen one after another: O(P) time.

Q. How can a library do it faster?

Using tree-based or recursive-doubling algorithms where many processes combine partial results at the same time, giving O(log P) steps.

Q. Where are prefix sums used?

Computing array offsets (e.g. in parallel sorting and compaction), cumulative histograms, load-balancing, and parallel algorithms like radix sort.

Q. Which operations can be used with MPI_Scan?

Any predefined reduction (MPI_SUM, MPI_PROD, MPI_MAX, MPI_MIN, ...) or a user-defined MPI_Op.

6. Common mistakes (examiners look for these)

- Rank 0 calling MPI_Recv (no previous rank) or the last rank calling MPI_Send - hangs.
- Sending the local value instead of the accumulated prefix to the next rank.
- Thinking Scan returns the result only on rank 0.
- Mixing up inclusive and exclusive prefix.

Problem 17: All-to-All Communication

1. Question (from the assignment)

Each MPI process generates data intended for every other process. Implement the communication using MPI_Alltoall and analyze its communication cost as the number of processes increases. Students should measure communication time for P = 2, 4, 8, 16 and different message sizes.

Sample Input

```
Number of processes: 4
P0: 0 1 2 3
P1: 10 11 12 13
P2: 20 21 22 23
P3: 30 31 32 33
Each process sends one value to every process.
```

Sample Output

```
After MPI_Alltoall:
P0 received: 0 10 20 30
P1 received: 1 11 21 31
P2 received: 2 12 22 32
P3 received: 3 13 23 33
```

2. Solution code

Compile / run: `mpicc p17.c -o p17 && mpirun -np 4 ./p17`; repeat with -np 2, 8, 16 (use `--oversubscribe` if you have fewer cores)

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    /* ---------- Part A: the 1-integer-per-destination demonstration ---------- */
    int *send_data = malloc(size * sizeof(int));
    int *recv_data = malloc(size * sizeof(int));
    for (int j = 0; j < size; j++) send_data[j] = rank * 10 + j;   /* element j goes to rank j */

    MPI_Alltoall(send_data, 1, MPI_INT, recv_data, 1, MPI_INT, MPI_COMM_WORLD);

    if (size <= 8) {                                    /* print in rank order */
        for (int r = 0; r < size; r++) {
            if (rank == r) {
                if (r == 0) printf("After MPI_Alltoall:\n");
                printf("P%d received:", rank);
                for (int j = 0; j < size; j++) printf(" %d", recv_data[j]);
                printf("\n");
                fflush(stdout);
            }
            MPI_Barrier(MPI_COMM_WORLD);
        }
    }
    free(send_data);
    free(recv_data);

    /* ---------- Part B: cost measurement for different message sizes ---------- */
    int counts[] = {1, 16, 256, 4096, 65536};      /* ints sent to EACH process */
    int nc = sizeof(counts) / sizeof(counts[0]);
    int reps = 10;

    for (int k = 0; k < nc; k++) {
        int m = counts[k];
        int *sbuf = malloc((size_t)size * m * sizeof(int));
        int *rbuf = malloc((size_t)size * m * sizeof(int));
        for (int i = 0; i < size * m; i++) sbuf[i] = rank;

        MPI_Alltoall(sbuf, m, MPI_INT, rbuf, m, MPI_INT, MPI_COMM_WORLD);   /* warm-up */
        MPI_Barrier(MPI_COMM_WORLD);
        double t0 = MPI_Wtime();
```

```
        for (int r = 0; r < reps; r++)
            MPI_Alltoall(sbuf, m, MPI_INT, rbuf, m, MPI_INT, MPI_COMM_WORLD);
        double local_t = (MPI_Wtime() - t0) / reps;

        double max_t;                                /* slowest process defines the cost */
        MPI_Reduce(&local_t, &max_t, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
        if (rank == 0)
            printf("P = %2d, %6d ints (%8d bytes) per destination : %.9f s\n",
                   size, m, m * (int)sizeof(int), max_t);
        free(sbuf);
        free(rbuf);
    }

    MPI_Finalize();
    return 0;
}
```

3. Step-by-step explanation (for the lab exam)

- **What Alltoall does:** it is a distributed matrix transpose. Each process has P blocks; block j goes to process j. After the call, block j of process i's receive buffer holds the block that process j had addressed to i.
- **Check with the sample:** send_data of P1 is [10 11 12 13]. Element 0 (10) goes to P0, element 1 (11) stays with P1, element 2 (12) goes to P2, element 3 (13) goes to P3. So P0 receives the first element of every process: 0 10 20 30. P1 receives the second element of every process: 1 11 21 31, and so on. Position in the receive buffer = rank of the sender.
- **Count arguments:** the *count* in MPI_Alltoall is the number of elements sent to *each* process (here 1), not the total. The buffer must have size x count elements.
- **Orderly printing:** a loop with barriers lets each rank print in turn, so the output looks like the sample (printing is skipped for P greater than 8 to keep output short).
- **Cost analysis:** each process sends P-1 messages and receives P-1 messages of size m. Total data moved over the network is P x (P-1) x m. So time grows with P (more messages) and with m (more data). For small m it is latency-dominated and grows roughly with P (or log P with Bruck's algorithm); for large m it is bandwidth-dominated and grows about linearly with P x m. It is the most expensive standard collective and often limits scalability, e.g. in FFTs and parallel sorting.
- **Measurement method:** warm-up call, barrier, many repetitions, then `MPI_Reduce` with `MPI_MAX` because the collective is only finished when the slowest process finishes. Run the program for P = 2, 4, 8, 16 and tabulate/plot time vs P for each size.

4. MPI calls used

- `MPI_Alltoall(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm)`
- `MPI_Reduce` with `MPI_MAX` for the time, `MPI_Wtime`, `MPI_Barrier`.

5. Viva questions and model answers

Q. What does MPI_Alltoall do?

Every process sends a distinct block of data to every process (including itself) and receives one block from every process. It behaves like a transpose of the data blocks across processes.

Q. Difference between Alltoall, Allgather and Scatter?

Scatter: one root sends different pieces. Allgather: each process contributes one block and everyone gets all blocks (same data to everyone). Alltoall: every process sends a different block to each other process.

Q. What does the count argument mean?

Number of elements sent to each destination (and received from each source), not the total buffer size.

Q. What is the communication complexity?

Each process sends and receives P-1 messages; total data is P(P-1)m. Time increases at least linearly with m and grows with P.

Q. Why is Alltoall expensive?

It needs all pairs of processes to communicate, which stresses network bisection bandwidth. Cost grows faster than for Bcast or Reduce.

Q. What if the blocks have different sizes?

Use MPI_Alltoallv with per-process counts and displacements.

Q. Why take the maximum time across processes?

The operation is not complete until the slowest process finishes, so the maximum represents the true cost.

6. Common mistakes (examiners look for these)

- Sending a buffer smaller than size x count elements - memory error.
- Hard-coding arrays of length 4 so the program only works for 4 processes.
- Thinking the receive buffer is ordered by destination instead of by source.
- Reporting only rank 0's time rather than the maximum.

Problem 18: Global Histogram with MPI_Allreduce

1. Question (from the assignment)

Divide a large dataset among MPI processes. Each process generates a local histogram, and MPI_Allreduce combines all local histograms into a global histogram.

Sample Input

```
Number of processes: 4
Values range from 0 to 9.
P0: 1 2 2 3
P1: 2 3 4 4
P2: 1 4 5 5
P3: 0 2 5 9
```

Sample Output

```
Final histogram after MPI_Allreduce:
Value Frequency
0 1
1 2
2 4
3 2
4 3
5 3
6 0
7 0
8 0
9 1
Total number of elements = 20     <-- see note below: the counts add up to 16
```

2. Solution code

Compile / run: `mpicc p18.c -o p18 && mpirun -np 4 ./p18`

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 4) {                    /* one data row per process */
        if (rank == 0) printf("Run with exactly 4 processes.\n");
        MPI_Finalize();
        return 1;
    }

    int data[4][4] = {{1,2,2,3}, {2,3,4,4}, {1,4,5,5}, {0,2,5,9}};  /* row r belongs to P r */
    int local_hist[10] = {0};
    int global_hist[10];

    for (int i = 0; i < 4; i++)
        local_hist[data[rank][i]]++;              /* value v -> bin v */

    /* sum the 10 bins element-wise across all processes; EVERY rank gets the result */
    MPI_Allreduce(local_hist, global_hist, 10, MPI_INT, MPI_SUM, MPI_COMM_WORLD);

    if (rank == 0) {
        int total = 0;
        printf("Final histogram after MPI_Allreduce:\n");
        printf("Value Frequency\n");
        for (int v = 0; v < 10; v++) {
            printf("%d %d\n", v, global_hist[v]);
            total += global_hist[v];
        }
        printf("Total number of elements = %d\n", total);
    }

    MPI_Finalize();
    return 0;
}
```

3. Step-by-step explanation (for the lab exam)

- **Local histogram:** an array of 10 counters, one per possible value 0-9. For each of its values v, a process does `local_hist[v]++`. P0 (1 2 2 3) gives bins 1:1, 2:2, 3:1.
- **Global histogram:** the global count of value v is the sum of the local counts of v over all processes. `MPI_Allreduce` with `MPI_SUM` and count 10 adds the arrays *element by element*.
- **Why Allreduce, not Reduce:** Allreduce leaves the full result on *every* process (equivalent to Reduce followed by Bcast). Here only rank 0 prints, but each rank could now use the global histogram, e.g. to normalise or to find the most common value.
- **Verify by hand:** value 0 appears once (P3); 1 twice (P0, P2); 2 four times (two in P0, one in P1, one in P3); 3 twice (P0, P1); 4 three times (two in P1, one in P2); 5 three times (two in P2, one in P3); 9 once (P3); 6, 7, 8 zero.
- **Important note about the sample output:** the assignment sheet says 'Total number of elements = 20', but 4 processes x 4 values = 16, and the listed frequencies add up to 1+2+4+2+3+3+1 = 16. The frequencies in the sheet are correct, only the total line is a typo. The program prints the total computed from the histogram (16). Mention this to the examiner if asked.
- **Scaling up:** for a large dataset, scatter the data (MPI_Scatter/Scatterv) so each rank gets a chunk, build the local histogram of that chunk, then Allreduce. Communication is only 10 integers regardless of dataset size, so this scales very well.

4. MPI calls used

- `MPI_Allreduce(sendbuf, recvbuf, count, type, op, comm)` - combine arrays element-wise and give the result to all ranks.
- (Large data) `MPI_Scatter / MPI_Scatterv` to distribute the dataset.

5. Viva questions and model answers

Q. What is a histogram?

A table of how many times each value (or each range of values, a bin) occurs in the data.

Q. What does MPI_Allreduce do?

Performs a reduction (here a sum) across all processes and makes the result available on every process. It is equivalent to MPI_Reduce followed by MPI_Bcast.

Q. Difference between MPI_Reduce and MPI_Allreduce?

Reduce: result only at root. Allreduce: result at all ranks, with no root argument.

Q. Why can the reduction be done element-wise on arrays?

The count argument (10) means 10 independent reductions: bin 0 of all processes is summed into bin 0, bin 1 into bin 1, etc.

Q. Why is this approach efficient?

Only 10 integers per process are communicated, independent of how much data each process has. The counting is done fully in parallel.

Q. What happens with a value outside 0-9?

local_hist[v] writes outside the array - undefined behaviour. In real code validate the input or compute the bin as (v - min) / bin_width.

Q. Why do all ranks need the same count and datatype in Allreduce?

Because it is a collective: all processes must supply buffers of matching size and type for the element-wise operation.

6. Common mistakes (examiners look for these)

- Calling MPI_Reduce and then reading the result on non-root ranks.
- Forgetting to initialise local_hist to zero.
- Using count 1 instead of 10 in Allreduce - only bin 0 is combined.
- Copying the '20' from the sheet - the real total is 16.

Rapid-Fire Viva Revision

Short answers you can say in one breath.

Q. What does MPI stand for?

Message Passing Interface - a standard for message-passing parallel programs on distributed-memory systems.

Q. Shared vs distributed memory?

Shared: threads see the same memory (OpenMP). Distributed: each process has its own memory and communicates by messages (MPI).

Q. What are the first and last MPI calls?

MPI_Init and MPI_Finalize.

Q. What is the default communicator?

MPI_COMM_WORLD - all processes started by mpirun.

Q. What is a rank?

The unique integer id of a process inside a communicator, from 0 to size-1.

Q. What is MPI_STATUS_IGNORE?

A constant telling MPI we do not need the status information of a receive.

Q. Blocking vs nonblocking?

Blocking returns when the buffer is safe; nonblocking returns at once and needs Wait/Test.

Q. What causes deadlock?

Circular waiting, e.g. both processes in MPI_Send before any receive. Fix: Sendrecv, reorder, or nonblocking.

Q. Point-to-point vs collective?

Point-to-point involves two processes with send/receive; collective involves all processes of a communicator in one call.

Q. Bcast vs Scatter?

Bcast: the same data to all. Scatter: different pieces to different processes.

Q. Reduce vs Allreduce?

Reduce: result at root. Allreduce: result at everyone.

Q. Gather vs Allgather?

Gather: collect at root. Allgather: collect and give the result to everyone.

Q. Scan vs Reduce?

Scan gives each rank a prefix result; Reduce gives one combined result at root.

Q. What is Alltoall?

Every process sends a different block to every other process (a block transpose).

Q. Why use MPI_Wtime?

To measure elapsed wall-clock time in seconds with high resolution.

Q. Latency vs bandwidth?

Latency: start-up time of a message. Bandwidth: data rate, bytes per second.

Q. How to avoid output order problems?

Gather results at root and print there, or take turns printing with barriers.

Q. Which process should do I/O?

Usually rank 0, then broadcast or scatter the data.

Q. What is speed-up?

T(serial) / T(parallel). Ideal = P; real speed-up is lower because of communication and imbalance.

Q. What is load imbalance?

Some processes get more work than others, so the rest wait idle at synchronisation points.